

1-Question: What is the default value assigned to array elements in C#?

The default value assigned to array elements in C# depends on their data type:

Numeric types (int, double): 0

Boolean types (bool): false

Reference types (string, objects): null

2-Question: What is the difference between Array.Clone() and Array.Copy()?

Array.Clone(): It creates a completely new array and copies all the elements into it. It returns an object, so you must cast it to your array type.

Array.Copy(): It copies elements from one existing array to another existing array. It gives you more control because you can choose how many elements to copy and from which index to start.

3-Question: What is the difference between GetLength() and Length for multi dimensional arrays?

Length: Returns the total number of all elements in the entire array (for a 3x3 array, it returns 9).

GetLength(dimension): Returns the number of elements in a specific dimension. For example, GetLength(0) gives the number of rows, and GetLength(1) gives the number of columns.

4-Question: What is the difference between Array.Copy() and Array.ConstrainedCopy()?

Array.Copy(): Copies elements normally. If an error happens during the copy process (like a wrong data type), it might stop halfway, leaving the new array partially changed.

Array.ConstrainedCopy(): It is much safer. It checks everything before starting. If any error happens, it cancels the operation and does not change the new array at all. It guarantees either a full successful copy or nothing.

5-Question: Why is foreach preferred for read-only operations on arrays?

The foreach loop is preferred because its syntax is simple and clean. It automatically loops through all the elements from start to finish, so you don't need to create an index variable or

worry about the array's length. This helps you avoid mistakes like index out-of-bounds errors. It also makes it clear to anyone reading your code that you only want to read the data, not change it.

6-Question: Why is input validation important when working with user inputs?

Input validation is very important because it protects your program from crashing. Users might enter the wrong type of data by mistake (like typing letters instead of numbers) or even try to enter bad data to break the system. By checking the input, you make sure your application runs safely, smoothly, and gives a good experience to the user.

7- Question: How can you format the output of a 2D array for better readability?

You can format the output by using a tab character (`\t`) to add equal spaces between the numbers so they align in straight columns. You should use `Console.Write()` to print the numbers of the same row next to each other, and then use `Console.WriteLine()` at the end of each row to move to a new line.

8-Question: When should you prefer a switch statement over if-else?

You should prefer a switch statement when you need to check a single variable against many specific, fixed values (like numbers or strings). It makes the code cleaner, easier to read, and sometimes faster than writing a long list of if-else conditions.

9-Question: What is the time complexity of `Array.Sort()`?

The average time complexity of `Array.Sort()` is $O(n \log n)$. Under the hood, it uses a very fast and efficient sorting algorithm called IntroSort to arrange the elements quickly.

10-Question: Which loop (for or foreach) is more efficient for calculating the sum of an array, and why?

Both loops are very fast, but the for loop is slightly more efficient. This is because the for loop accesses the elements directly using their index. The foreach loop is a bit slower because it creates a hidden object (called an enumerator) to move through the array, but it is much easier to read and write.

11-LinkedIn article about loops statements in Csharp :

Every C# developer uses loops daily, but are you using the right one for your specific scenario? Here is a quick breakdown of the four main loop types in C# and when to use them.

The for Loop Best when you know exactly how many iterations you need. It gives you full control over the index, making it highly efficient for accessing array elements directly or modifying them in place.

The foreach Loop The absolute favorite for read-only operations on collections. It hides the complexity of the index, resulting in cleaner, less error-prone code. While slightly less efficient than a traditional for loop due to the enumerator overhead, its readability makes it the standard choice for iterating through lists and arrays in .NET.

The while Loop Ideal when the number of iterations is unknown and depends on a specific condition being met. It evaluates the condition before executing the code block, meaning it might not run at all if the condition is false from the start.

The do-while Loop Similar to the while loop, but it guarantees that the code block will execute at least once. This is incredibly useful for scenarios like user input validation (defensive coding), where you must prompt the user first before checking if their input is valid.

12-Write a program that takes an integer input from the user (1-7) and prints the corresponding day using the enum.

If the user enters a number outside the range of 1 to 7, the program will not crash. Enum.Parse will accept the number, but it will just print the number itself (for example, 8) because there is no matching name for it in the enum.

12-What happens if the user enters a value outside the range of 1 to 7?

The program will not crash or show an error. The Enum.Parse method will accept the input, but since there is no matching name for that number in the enum, it will simply print the number itself (for example, 8).

